



UNIVERSITY
OF HULL

Live Software Updates in a Real-Time System

Nathan Rignall

This dissertation is submitted on April, 2024

Abstract

My abstract ...

Acknowledgements

My acknowledgements ...

Contents

1	Background	6
2	Survey	8
2.1	Review	8
2.2	Case Study	8
2.2.1	Control Systems	8
2.2.2	Dynamic Updates	8
2.2.3	Disruption-free	8
2.2.4	A model	8
2.2.5	Grid-connected	9
3	Runtime	10
4	Requirements	12
5	Design	14
6	Testing	15
7	Evaluation	16
8	Conclusion	17
	References	19
A	Extra Information	20

Chapter 1

Background

Real-Time A real-time system is defined as a system which requires correct logical operation with completion a configured deadline. In contrast to a non-real-time system which has no configured deadlines. ?

These real-time systems can be categorised into hard and soft. Hard real-time systems contain strict timing constraints, where a response must be calculated in a specified time-frame and a missed deadline could result in a catastrophic outcome. Soft real-time systems also have timing constraints, but a missed deadline should not result in a catastrophic outcome.

Typical hard real-time systems include aircraft or robotic control computers and soft real-time systems include media players or gaming computers. Soft real-time systems can be used in control applications, but the soft timing must be considered in the system design.

Control Systems A control system Differential equation

Operating Systems An Operating System (OS) is a piece of software that controls the execution of application programs on a system. Usually this involves managing resource allocations, such as a processor execution time, memory usage and generic inputs/outputs. ?

In order to develop a real-time applications with an OS, a RTOS (Real-Time Operating System) is usually required. This is to ensure the time deadlines of an application are enforced.

Hard real time operating system are designed to run applications with absolute timing Worst-Case Execution Time Assigned a relative deadline, all tasks must complete in their deadline. Modelled mathematically to ensure that all tasks run to completion

Soft Real Time

Define the maximum delay after a deadline

Scheduling Memory

Mixed Criticality

Middleware

Chapter 2

Survey

2.1 Review

Analysis of existing dynamic software updating techniques for safe and secure industrial control systems

Challenges in Dynamic Software Updating

A survey of dynamic software updating

2.2 Case Study

2.2.1 Control Systems

Live Seamless Firmware Upgrade in Real Time Control Systems

2.2.2 Dynamic Updates

Dynamic Software Updates for Real-Time Systems.

2.2.3 Disruption-free

Disruption-free software updates in automation systems

2.2.4 A model

A Model for Updating Real-Time Applications

2.2.5 Grid-connected

Enhanced Uptime and Firmware Cybersecurity for Grid-Connected Power Electronics

Chapter 3

Runtime

To select the most optimal RTOS a series of benchmarking activities were carried out

What is Linux

How containers work

What is Xen

How rt-tests work

Results

Bitbake

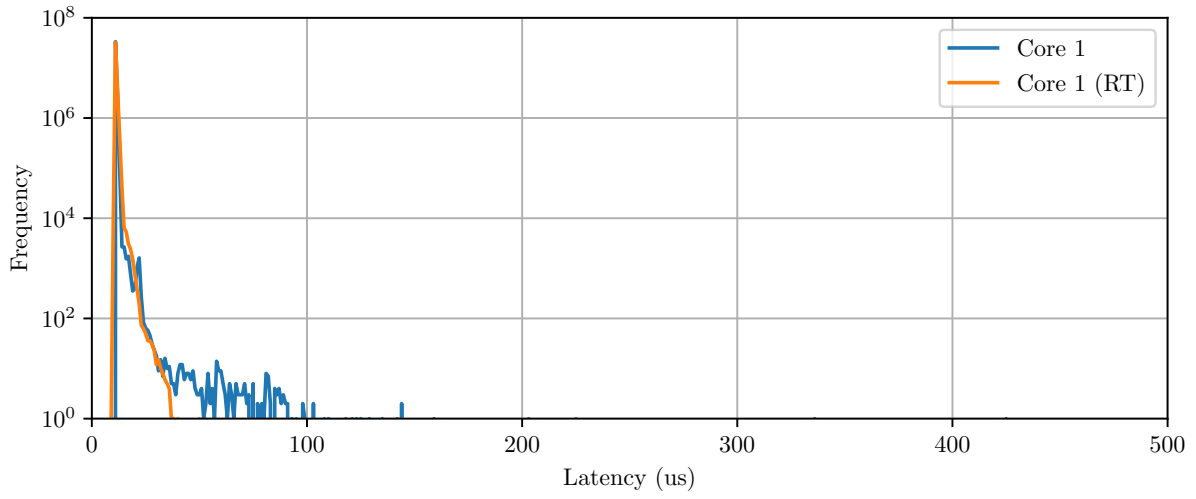


Figure 3.1: Latency Graph

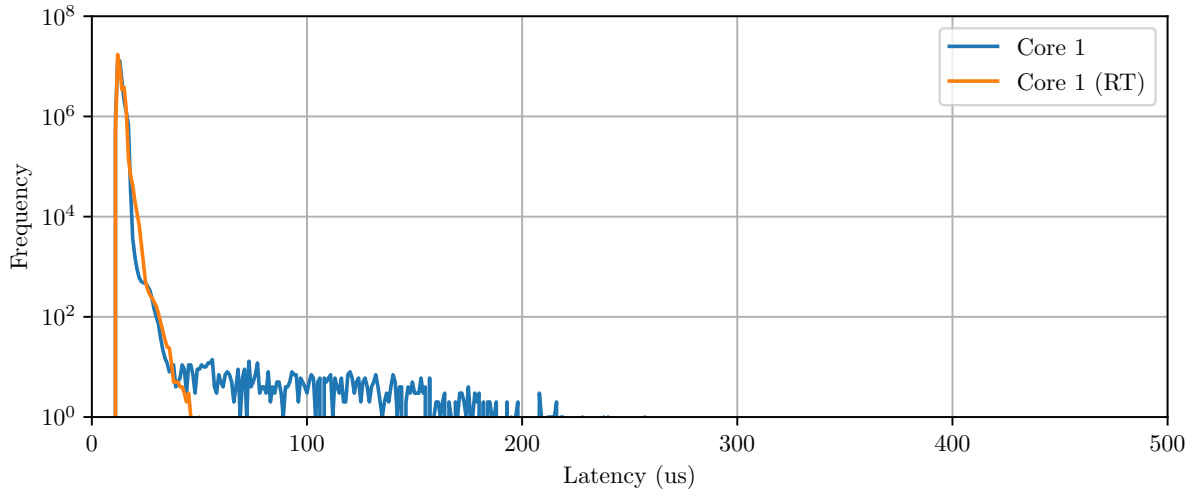


Figure 3.2: Latency Graph

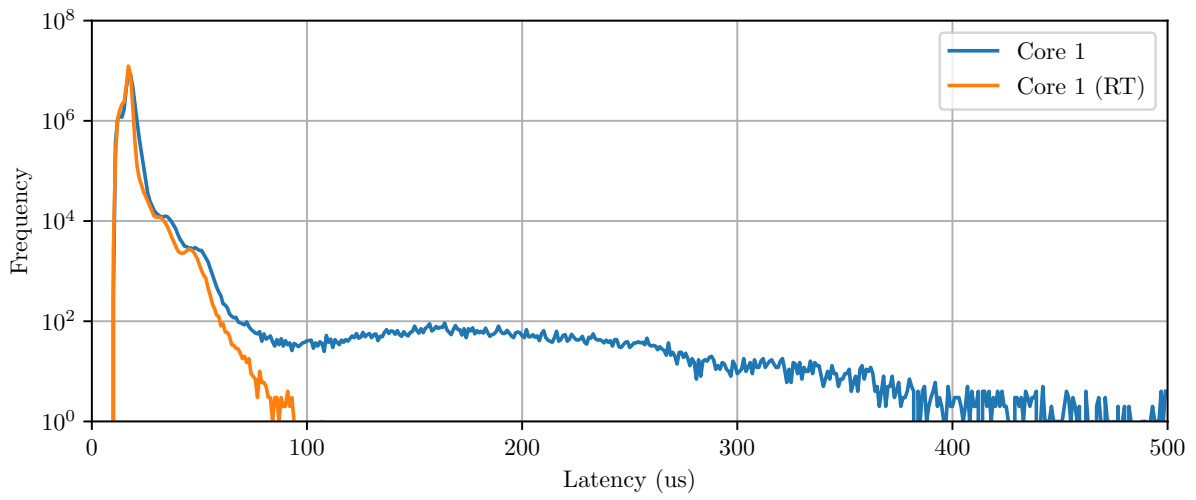


Figure 3.3: Latency Graph

Chapter 4

Requirements

The project's primary objective is to develop multiple system architectures to enable live software updates in a running control real-time system.

Framework for developers to run software components which can be dynamically updated

Example application that uses one differential equation and requires its state to be maintained with a plant for the example application to test its functionality

Testing harness that can measure th

High-Level During operation

The software infrastructure must provide the functionality to run software components with a cyclic schedule allow users to create/start/stop/remove software components during operation allow users to change the configured cyclic schedule during operation

provide the functionality to allow components to communicate using messages passing allow users to configure communications between components using routes and channels allow users to add/remove routes between software components during operation

Low-Level create - loads the software component binary - starts the software component

start - marks the software component as ready to run

stop - marks the software components as not ready to run

stop -

The example application must maintained

The plant must model a mass falling due to gravity

Chapter 5

Design

Chapter 6

Testing

Chapter 7

Evaluation

Chapter 8

Conclusion

References

Appendix A

Extra Information

Some more text ...